MINI PROJECT REPORT

CAMPUS ROUTE FINDER USING GRAPHS

Submitted By: Lakshya

Roll Number: 2556470

Course: BTech CSE A

Subject: Data Structures and Algorithms

1. INTRODUCTION

The Campus Route Finder is a mini project developed using the concepts of Graphs in Data Structures and Algorithms. It represents different locations of a campus as vertices and the paths connecting them as edges.

The project helps a user find the shortest route between two locations using Breadth-First Search (BFS). It also uses Depth-First Search (DFS) to find reachable locations, check whether a destination can be reached, and identify disconnected locations through connected components. The graph is stored using an adjacency list, and an adjacency matrix is also implemented so that both representations can be compared.

The project is implemented in the C programming language using a menu-driven interface.

2. OBJECTIVES

The main objectives of this project are:

- To understand the concept of graphs and their representations.
- To represent campus locations and the paths between them using an adjacency list and an adjacency matrix.
- To find the shortest route (minimum number of edges) between two locations using BFS.
- To find reachable locations and check reachability using DFS.
- To identify disconnected locations and connected components.
- To explain and compare the time and space complexity of BFS and DFS for both representations.
- To verify the program using test cases with expected and actual outputs.

3. PROBLEM STATEMENT

Finding a route between two places on a campus can be difficult when several paths are available, and some places may not be connected to the rest of the campus. This project represents campus locations as a graph and allows the user to enter a source and a destination. The program displays the route with the minimum number of edges when a route exists, and reports clearly when no route is available.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Programming Language: C

- Compiler: GCC
- Editor: Visual Studio Code with the integrated terminal
- Operating System: macOS (the program also runs on Windows and Linux)

Hardware Requirements

- Computer or laptop
- Basic memory and storage
- Keyboard and display

5. DATA STRUCTURE USED

A graph consists of vertices and edges. In this project the vertices represent campus locations and the edges represent direct paths between two locations. The graph is undirected (every path can be used in both directions) and unweighted (every path counts as one edge).

The program contains nine locations:

No.	Location	No.	Location
0	Main Gate	5	Auditorium
1	Library	6	Parking
2	Block 1	7	Lab
3	Block 2	8	Sports Ground
4	Cafeteria		

The initial graph has the following nine paths (edges):

Path	Between
1	Main Gate - Library
2	Main Gate - Parking
3	Library - Block 1
4	Library - Cafeteria
5	Block 1 - Block 2
6	Block 1 - Lab
7	Block 2 - Auditorium
8	Cafeteria - Auditorium
9	Parking - Lab

Sports Ground has no path to any other location, so it is a disconnected location. This is used to demonstrate an unreachable destination.

5.1 Adjacency List

The adjacency list is the main representation. It is an array of linked lists. Each location has a linked list that stores the locations directly connected to it. Its space complexity is O(V + E), so it is efficient for sparse graphs such as a campus map.

5.2 Adjacency Matrix

The adjacency matrix is a V x V two-dimensional array. A value of 1 at position [i][j] means there is a direct path between location i and location j, and 0 means there is none. Because the graph is undirected, the matrix is symmetric. Its space complexity is $O(V^2)$. A direct connection between two locations can be checked in O(1) time.

6. ALGORITHMS USED

6.1 Breadth-First Search (BFS)

BFS explores the graph level by level using a queue. It visits all neighbouring locations before moving to locations that are farther away. Because every edge has the same cost, the first time BFS reaches the destination it has used the minimum number of edges.

Steps:

1. Mark the source location as visited and add it to the queue.
2. Remove a location from the front of the queue.
3. For each unvisited neighbour, mark it visited, store the current location as its parent, and add it to the queue.
4. Repeat until the destination is reached or the queue becomes empty.
5. If the destination was never visited, display that no route is available.
6. Otherwise, follow the parent array from the destination back to the source, reverse it, and display the path.

6.2 Depth-First Search (DFS)

DFS explores one path as far as possible before returning to explore another path. This implementation uses recursion (the call stack acts as the stack) and a visited array.

Steps:

1. Start from the selected location and mark it visited.
2. Recursively visit each unvisited neighbouring location.
3. After the traversal, all visited locations are reachable from the starting location.
4. Locations that were not visited are displayed as disconnected from the starting location.

6.3 Reachability Check

To check whether a destination is reachable from a source, DFS is run from the source and the visited status of the destination is checked.

6.4 Connected Components

DFS is started from every location that has not been visited yet. Each such DFS call finds one connected component, that is, a group of locations that can reach each other. If there is more than one component, the campus has disconnected locations.

7. SYSTEM IMPLEMENTATION

The program is implemented in C and provides the following menu options:

Option	Function
1	Display campus locations
2	Display adjacency list
3	Display adjacency matrix
4	Find shortest route (BFS)
5	Reachable locations with DFS traversal order
6	Check if destination is reachable (DFS)
7	Show connected components
8	Add a new path
9	Run test cases
10	Show complexity comparison
0	Exit

For route finding, the user enters the source and destination location numbers. The program checks that the numbers are valid (0 to 8) and then displays the route, the number of edges, or a message that no route is available. The program frees all dynamically allocated adjacency list nodes before exiting.

8. TEST CASES

Option 9 of the program runs five test cases automatically and compares the expected output with the actual output. The results from the run shown in Screenshot 5 are given below.

Test	Description	Input	Expected Output	Actual Output	Result
1	Direct route	Main Gate to Library	Main Gate -> Library	Main Gate -> Library	PASS
2	Multi-stop route	Main Gate to Auditorium	Main Gate -> Library -> Cafeteria -> Auditorium	Main Gate -> Library -> Auditorium	FAIL (see note)
3	Multi-stop route	Parking to Cafeteria	Parking -> Main Gate -> Library -> Cafeteria	Parking -> Main Gate -> Library -> Cafeteria	PASS
4	Unreachable destination	Main Gate to Sports Ground	No route	No route	PASS
5	Source and destination same	Block 1 to Block 1	Block 1	Block 1	PASS

Summary: 4 out of 5 test cases passed.

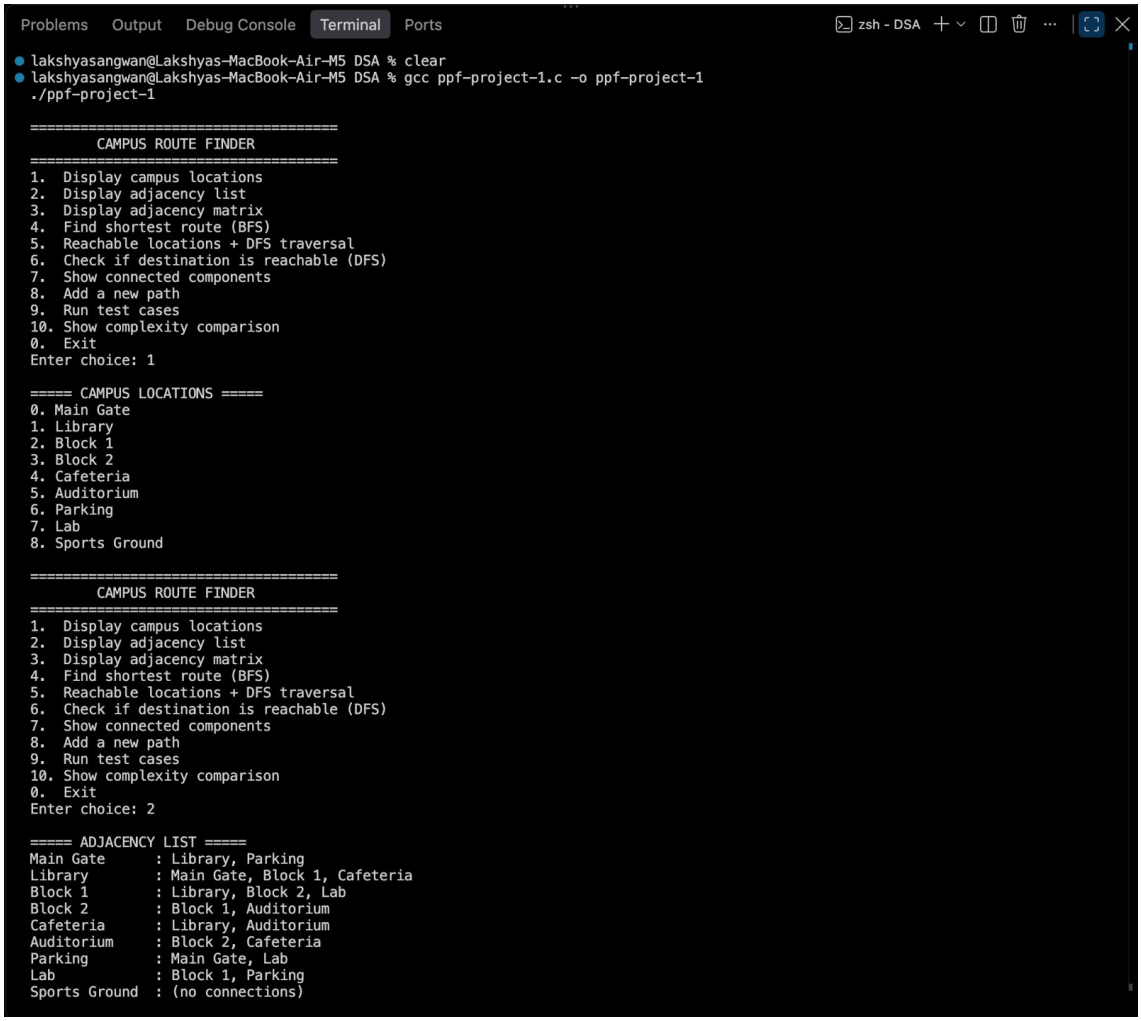
Note on Test 2: The expected output of Test 2 is written in the program for the original campus graph. Before the test cases were run, a new path between Library and Auditorium was added using option 8 (Screenshot 4). After this path was added, the shortest route from Main Gate to

Auditorium became Main Gate -> Library -> Auditorium with 2 edges, which is shorter than the original 3-edge route. The actual output is therefore the correct shortest route for the updated graph, and the mismatch is only because the expected value was fixed for the original graph. Before the path was added, BFS from Main Gate (0) to Auditorium (5) gave Main Gate -> Library -> Cafeteria -> Auditorium with 3 edges (Screenshot 3), which matches the expected output.

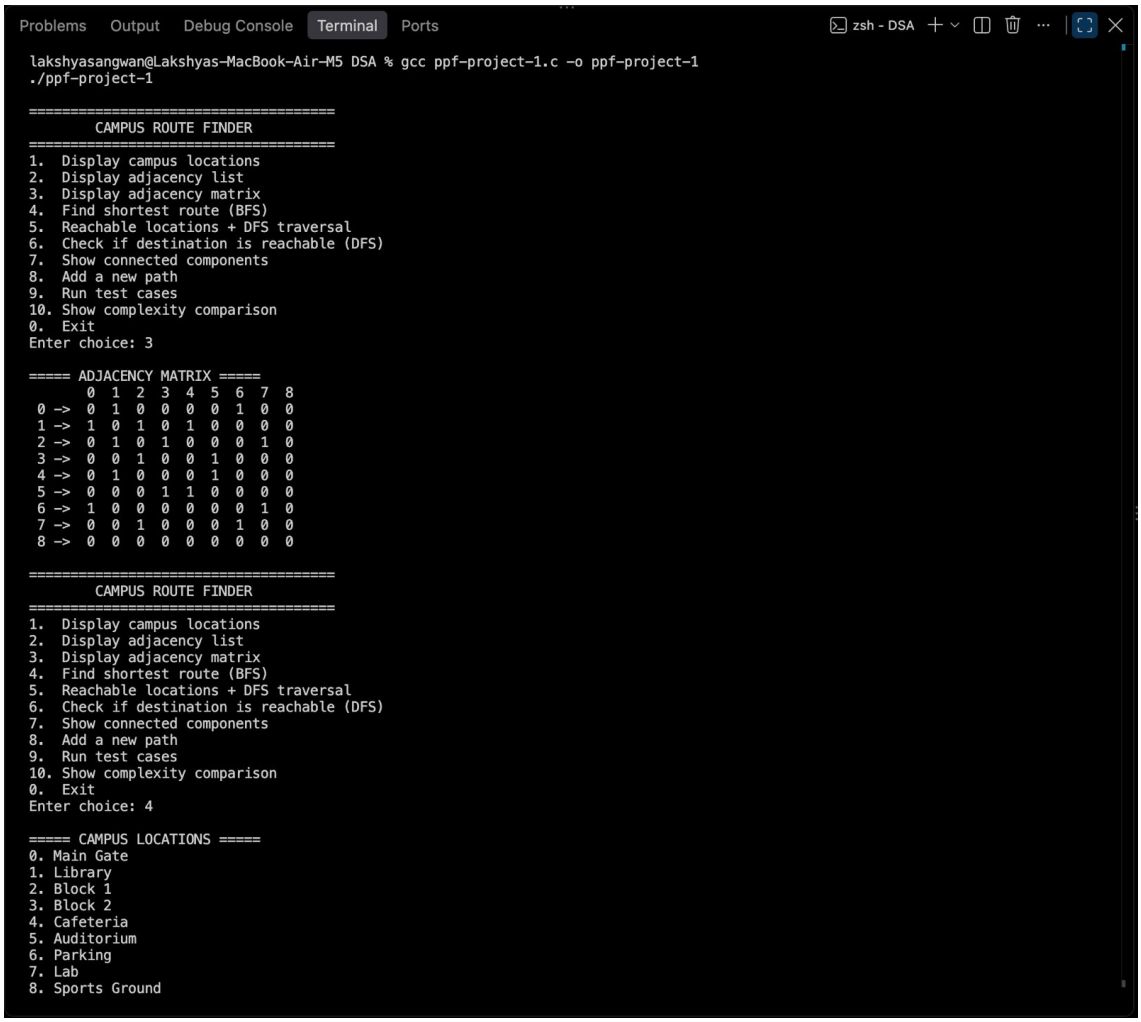
The reachability check using DFS from Auditorium to Main Gate also reported that Main Gate is reachable (Screenshot 3), and the connected components output showed Sports Ground as a separate component (Screenshot 4).

9. OUTPUT SCREENSHOTS

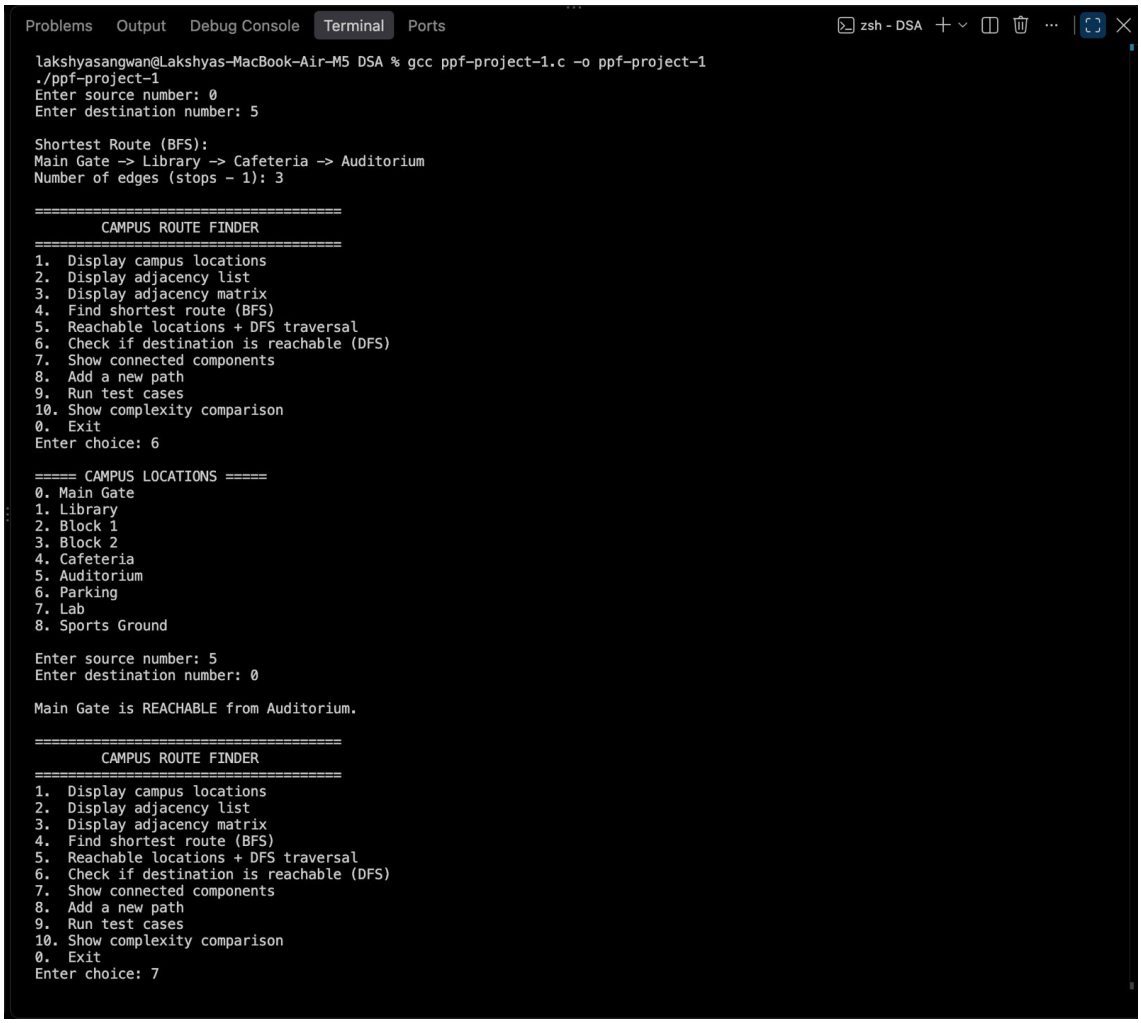
Screenshot 1: Main menu, campus locations (option 1) and adjacency list (option 2)



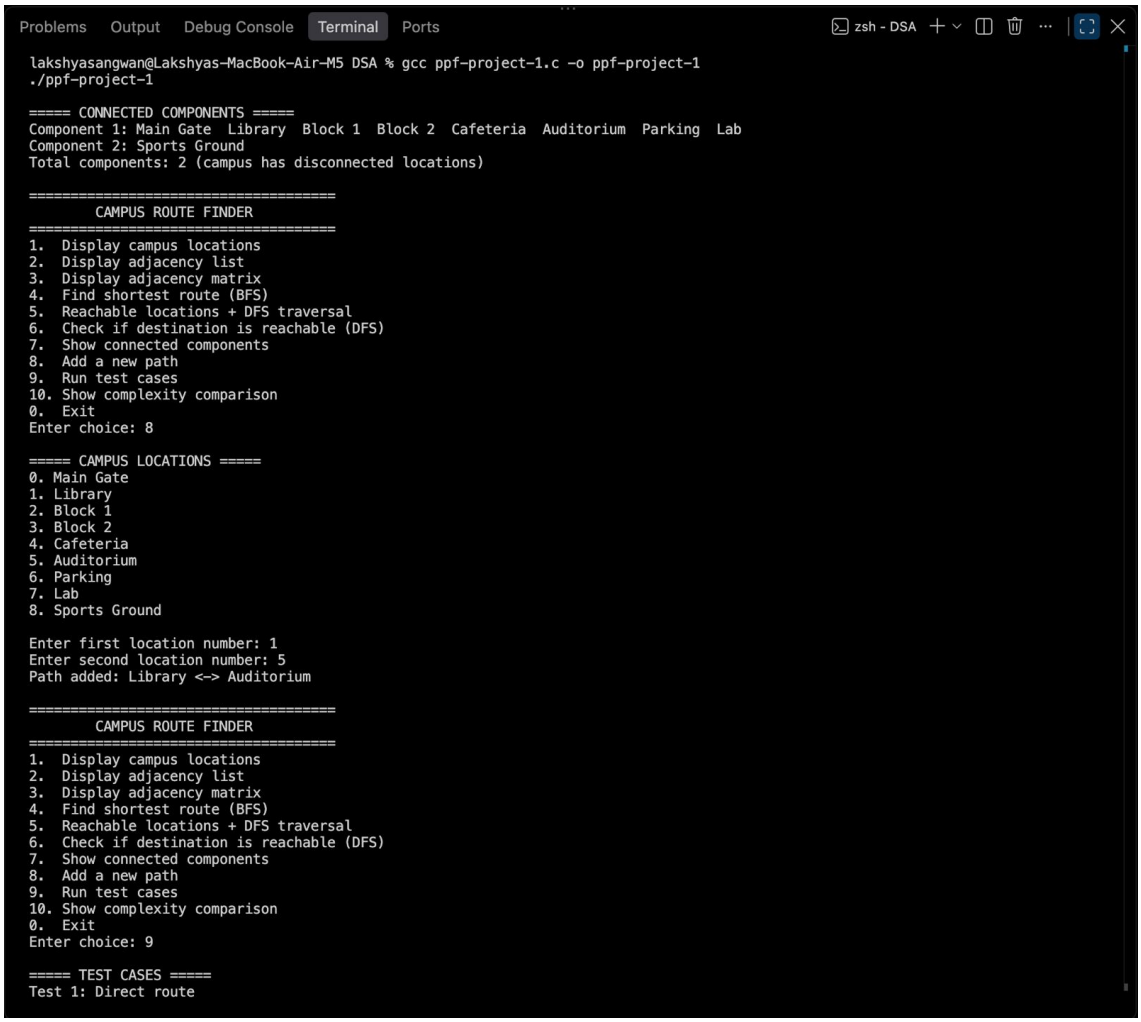
Screenshot 2: Adjacency matrix (option 3)



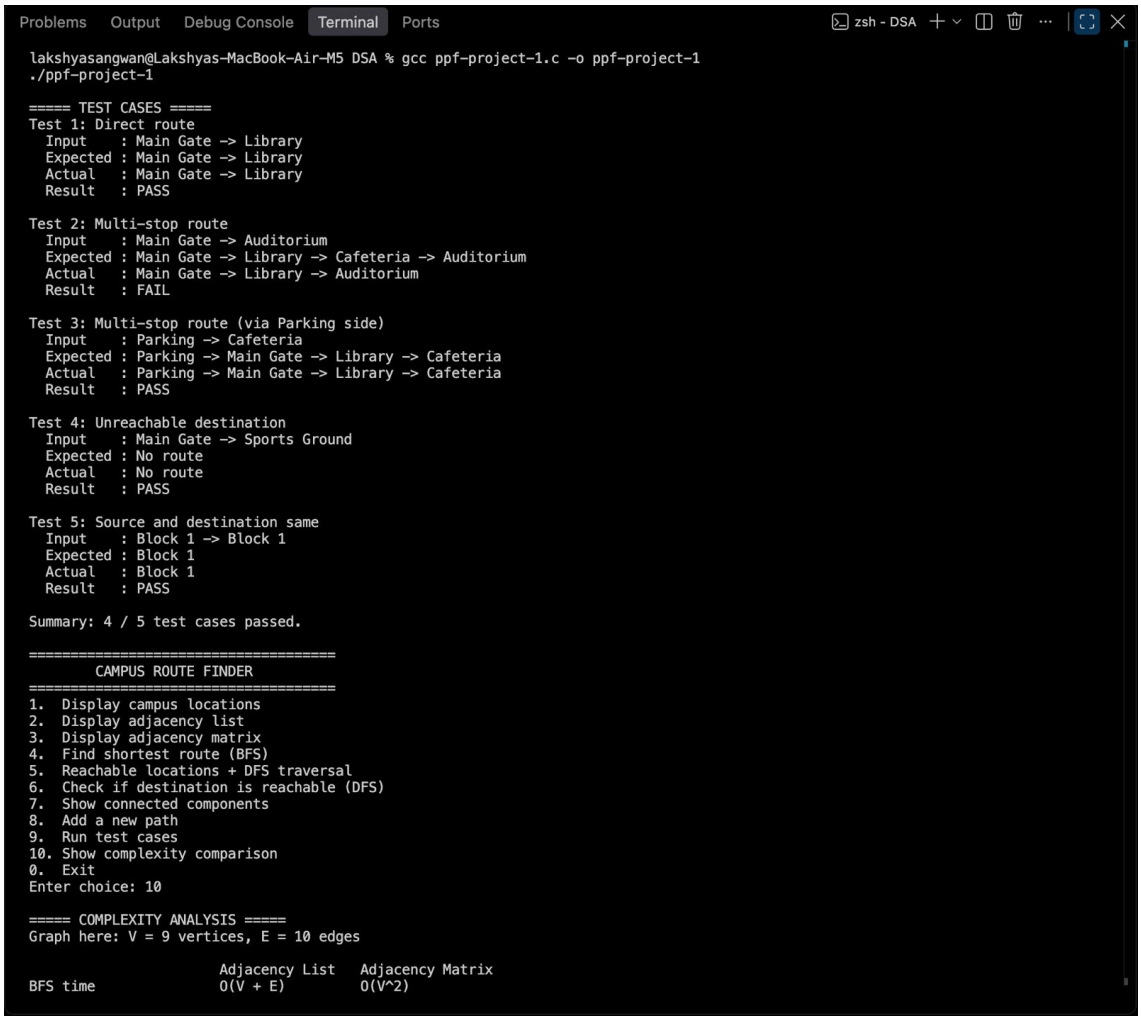
Screenshot 3: Shortest route using BFS (option 4) and reachability check using DFS (option 6)



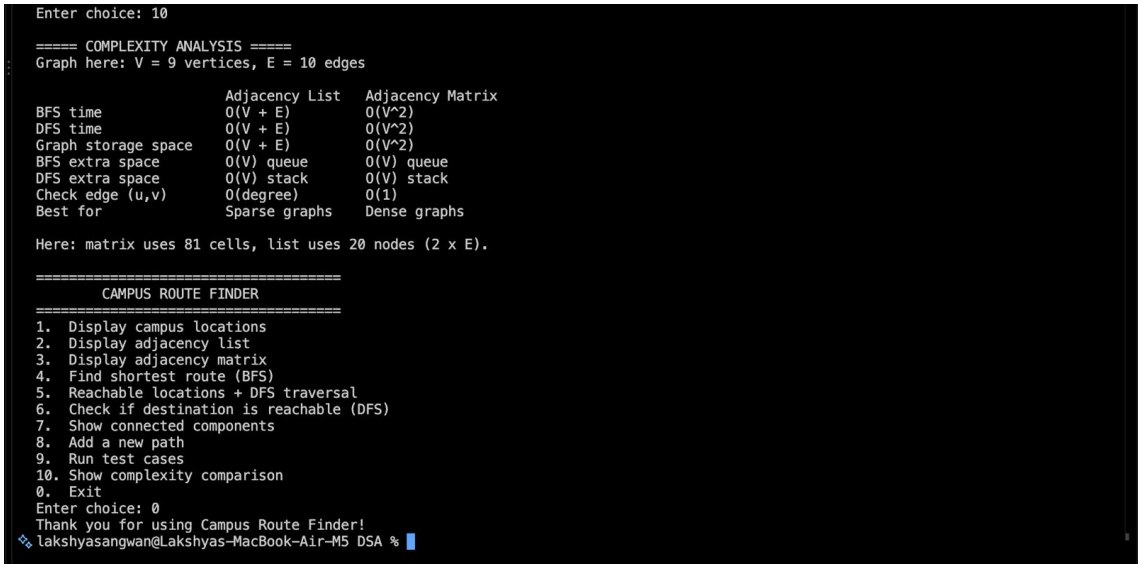
Screenshot 4: Connected components (option 7) and adding a new path (option 8)



Screenshot 5: Test cases (option 9) and start of complexity analysis (option 10)



Screenshot 6: Complexity analysis (option 10) and exit (option 0)



10. TIME AND SPACE COMPLEXITY

Let V represent the number of vertices and E represent the number of edges. In this project V = 9. The initial graph has E = 9, and after adding one path during the run shown in the screenshots E = 10.

Operation	**Adjacency List**	**Adjacency Matrix**
BFS time complexity	O(V + E)	$O(V^2)$
DFS time complexity	O(V + E)	$O(V^2)$
Graph storage space	O(V + E)	$O(V^2)$
BFS extra space	O(V) for queue, visited and parent arrays	O(V) for queue, visited and parent arrays
DFS extra space	O(V) for recursion stack and visited array	O(V) for recursion stack and visited array
Check edge (u, v)	O(degree of u)	O(1)
Best suited for	Sparse graphs	Dense graphs

Why O(V + E) with an adjacency list: each vertex is visited once, and each edge is examined a constant number of times (twice in an undirected graph) while the neighbour lists are scanned.

Why $O(V^2)$ with an adjacency matrix: to find the neighbours of a vertex, the whole row of V entries must be scanned, and this is done for each of the V vertices.

Comparison: The campus graph is sparse because each location is connected to only a few others. In the program output (Screenshot 6), the adjacency matrix uses 81 cells while the adjacency list uses only 20 nodes (2 x E). Therefore the adjacency list is the better choice for this project.

11. ADVANTAGES

- Simple and easy-to-use menu-driven interface.
- Demonstrates both adjacency list and adjacency matrix representations.
- Finds the shortest route (minimum edges) in an unweighted graph using BFS.
- Identifies reachable locations, disconnected locations and connected components using DFS.
- Allows new paths to be added while the program is running.
- Includes built-in test cases with expected and actual outputs.
- Validates location numbers before processing and frees allocated memory before exit.

12. LIMITATIONS

- The initial campus locations and connections are predefined in the program.
- The program does not calculate real-world distances or travel times.
- All paths are treated as equal (unweighted), so the shortest route means the fewest edges, not the shortest distance.
- Locations cannot be added or removed while the program is running; only new paths can be added.
- The expected outputs of the built-in test cases are fixed for the original graph.

13. FUTURE SCOPE

The project can be improved by:

- Allowing users to add or remove locations and routes.
- Assigning distances or weights to the paths.
- Finding minimum-distance routes using Dijkstra's algorithm.
- Saving and loading the campus map from a file.
- Developing a graphical interface or campus map.

14. CONCLUSION

The Campus Route Finder demonstrates how graphs can be used to represent campus locations and their connections. BFS with parent tracking is used to find the route with the minimum number of edges in an unweighted graph, while DFS is used to find reachable locations, check reachability and identify connected components. The comparison of the adjacency list and the adjacency matrix shows that the adjacency list is more efficient for a sparse graph like a campus map.

This project provided practical experience with graph representation, traversal algorithms, queues, recursion, linked lists, dynamic memory and menu-driven programming in C.

15. REFERENCES

1. Data Structures and Algorithms course notes.
2. C Programming Language reference materials.
3. Graph concepts: adjacency list, adjacency matrix, Breadth-First Search and Depth-First Search.